Object-Oriented Design Interview

Info overview

Design a Parking Lot

Design a Vending Machine

Design a Cache

Design a Online Ticket Booking System

Design a Ride Sharing Service

Design an Elevator System

Design an Online Chess System

Design a Stock Exchange

Design a Pub/Sub System

Design a Recommendation Engine

04
Design a Parking Lot

In this chapter, we explore the object-oriented design of a Parking Lot system, one of the most popular questions in technical interviews. The problem is deceptively simple but provides a comprehensive solution for efficiently managing a parking lot. It encounters various processes, including vehicle entry, exit, and spot allocation, while also providing accurate information about parking lot occupancy and generating parking tickets.

To tackle this system, we first need to clarify the requirements.

Parking Lot

Requirements Gathering

The first step in designing a parking lot system is to clarify the requirements and define the scope. Interim an example of a conversation between a candidate and an interviewer might unfold:

"Designing parking lot management system is a complex task, so let's focus on the core functionality. For this project, we'll assume that a spot suitable for any vehicle (bike, car and truck) is taken when you prepare to leave, you unpaid your ticket at the exit, the system calculates the fee based on the time spent, and the system should track the number of available spots for different vehicle types. We'll also assume that vehicles can, for simplicity, enter and exit the parking lot without any issues. First, let's begin by creating a system that handles all fees."

Requirements clarification

In this step, we are clarifying questions to narrow down the list of requirements, understand the constraints, and define the problem scope. Here's a conversation between a candidate and an interviewer might unfold:

Here's an example of how a conversation between a candidate and an interviewer might unfold:

Candidate: What types of vehicles are supposed to park in the parking lot?
Interviewer: These types of vehicles should be supported: motorcycles, cars, and trucks.

Candidate: How do we determine the parking fee for different vehicles?
Interviewer: The parking lot supports three types of parking spots: compact, regular spots, and oversized.

Candidate: Do we need to support different parking rates for different vehicle types?
Interviewer: Yes, we'll assume a flat rate per hour for each type of vehicle, whereas, we'll assume for simplicity.

Candidate: Do parking tickets issued for vehicles upon entry and charged at the exit?
Interviewer: Yes, the system should issue a ticket upon vehicle entry and charge when a vehicle exists. On exit, the system calculates the fee based on the time spent and bill the driver, then marks the spot as vacant.

Candidate: How are vacant spots allocated?
Interviewer: Vacant spots are based on parking duration and vehicle size, with cars varying depending on the time of day.

Requirements

As we clarify clarifying questions, we should delve into the requirements for the problem. Putting the key requirements in writing will help us avoid ambiguity and ensure clarity, as there is nothing worse than spending time on something we aren't supposed to do.

Here are the key functional requirements we've identified:

- The parking lot supports three types of parking spots: compact, regular, and oversized spots.
- The system must accurately track the number of available spots for each vehicle type.
- Customers can park their vehicle in spots assigned based on vehicle size.
- The system should issue a parking ticket upon vehicle entry and calculate the fee at the entry point and pay at the based on duration, oversized, and regular spots.
- The system should accurately track the number of available spots in real-time.

Below are the non-functional requirements:

- The system must scale to support large parking lots with many spots and vehicles.
- The system must reliably track spot assignments and issue tickets in error-free operations.

With these requirements in mind, we can identify the core objects.

Identify Core Objects

Now that we have clarified the requirements, we can identify the core objects that the system will interact with.

- **Vehicle**: This represents the various types of vehicles that the parking lot system must handle, including cars, trucks, and motorcycles. Each vehicle has attributes such as a unique identifier, size, and the type of vehicle, which are important for assigning parking spots and calculating fees.
- **ParkingSpot**: This represents individual parking spots in the parking lot. It's the physical space where a vehicle parks, and it has attributes such as its size, location, and whether it is occupied or vacant.
- **ParkingLot**: This represents the entire parking lot and allows managing the assignment, tracking, and release of ParkingSpot instances. It ensures it efficiently gets the right spot by checking availability based on size, and updates the system when a vehicle enters or leaves, maintaining accurate records of occupied and vacant spots.
- **ParkingLot**: This acts as a central hub, coordinating the overall operation of the parking lot. It manages the assignment of vehicles to spots, tracks the overall availability, and calculates fees. It keeps the right ParkingSpot by delegating tasks such as spot allocation to the appropriate ParkingSpot instances, maintaining a clear separation of concerns and ensuring the efficient and cost-effective handling of parking operations.

Identify the core objects

Below are the core relationships:

- The parking lot supports various types of parking spots (compact, regular spots, and oversized).
- Each parking spot can accommodate one vehicle at a time, ensuring efficient space utilization.
- The system assigns vehicles to spots based on their size, with larger vehicles requiring more spots.
- The ParkingLot class acts as the central hub, managing the overall operation and coordinating between vehicles and parking spots.

Below are the core functional requirements:

- The system must accurately track the number of available spots for each vehicle type.
- The system must reliably track spot assignments and issue tickets in error-free operations.

With these requirements, we can now identify the core objects.

Identify Core Objects

Now that we have clarified the requirements, we can identify the core objects that the system will interact with.

- **Vehicle**: This represents the various types of vehicles that the parking lot system must handle, including cars, trucks, and motorcycles. Each vehicle has attributes such as a unique identifier, size, and the type of vehicle, which are important for assigning parking spots and calculating fees.
- **ParkingSpot**: This represents individual parking spots in the parking lot. It's the physical space where a vehicle parks, and it has attributes such as its size, location, and whether it is occupied or vacant.
- **ParkingLot**: This represents the entire parking lot and allows managing the assignment, tracking, and release of ParkingSpot instances. It ensures it efficiently gets the right spot by checking availability based on size, and updates the system when a vehicle enters or leaves, maintaining accurate records of occupied and vacant spots.
- **ParkingLot**: This acts as a central hub, coordinating the overall operation of the parking lot. It manages the assignment of vehicles to spots, tracks the overall availability, and calculates fees. It keeps the right ParkingSpot by delegating tasks such as spot allocation to the appropriate ParkingSpot instances, maintaining a clear separation of concerns and ensuring the efficient and cost-effective handling of parking operations.

Identify the core objects

Below are the core relationships:

- The parking lot supports various types of parking spots (compact, regular spots, and oversized).
- Each parking spot can accommodate one vehicle at a time, ensuring efficient space utilization.
- The system assigns vehicles to spots based on their size, with larger vehicles requiring more spots.
- The ParkingLot class acts as the central hub, managing the overall operation and coordinating between vehicles and parking spots.

Below are the core functional requirements:

- The system must accurately track the number of available spots for each vehicle type.
- The system must reliably track spot assignments and issue tickets in error-free operations.

With these requirements, we can now identify the core objects.

Identify Core Objects

As we clarify clarifying questions, we should delve into the requirements for the problem. Putting the key requirements in writing will help us avoid ambiguity and ensure clarity, as there is nothing worse than spending time on something we aren't supposed to do.

Here are the key functional requirements we've identified:

- The parking lot supports three types of parking spots: compact, regular, and oversized spots.
- The system must accurately track the number of available spots for each vehicle type.
- Customers can park their vehicle in spots assigned based on vehicle size.
- The system should issue a parking ticket upon vehicle entry and calculate the fee at the entry point and pay at the based on duration, oversized, and regular spots.
- The system should accurately track the number of available spots in real-time.

Below are the non-functional requirements:

- The system must scale to support large parking lots with many spots and vehicles.
- The system must reliably track spot assignments and issue tickets in error-free operations.

With these requirements, we can now identify the core objects.

Identify Core Objects

Now that we have clarified the requirements, we can identify the core objects that the system will interact with.

- **Vehicle**: This represents the various types of vehicles that the parking lot system must handle, including cars, trucks, and motorcycles. Each vehicle has attributes such as a unique identifier, size, and the type of vehicle, which are important for assigning parking spots and calculating fees.
- **ParkingSpot**: This represents individual parking spots in the parking lot. It's the physical space where a vehicle parks, and it has attributes such as its size, location, and whether it is occupied or vacant.
- **ParkingLot**: This represents the entire parking lot and allows managing the assignment, tracking, and release of ParkingSpot instances. It ensures it efficiently gets the right spot by checking availability based on size, and updates the system when a vehicle enters or leaves, maintaining accurate records of occupied and vacant spots.
- **ParkingLot**: This acts as a central hub, coordinating the overall operation of the parking lot. It manages the assignment of vehicles to spots, tracks the overall availability, and calculates fees. It keeps the right ParkingSpot by delegating tasks such as spot allocation to the appropriate ParkingSpot instances, maintaining a clear separation of concerns and ensuring the efficient and cost-effective handling of parking operations.

Identify the core objects

Below are the core relationships:

- The parking lot supports various types of parking spots (compact, regular spots, and oversized).
- Each parking spot can accommodate one vehicle at a time, ensuring efficient space utilization.
- The system assigns vehicles to spots based on their size, with larger vehicles requiring more spots.
- The ParkingLot class acts as the central hub, managing the overall operation and coordinating between vehicles and parking spots.

Below are the core functional requirements:

- The system must accurately track the number of available spots for each vehicle type

```
        .SMALL_VEHICLE_RATE;
    long hours = (long) Math.ceil(1.0 * durationMinutes / 60);
    fare = fare.add(rate.multiply(ticket.calculateParkingDuration()));
    return fare;
}
}
```

calculateFare(Ticket ticket, BigDecimal inputFare): Provides the foundational cost for the parking session, reflecting size-based pricing.

The concrete class PeakHoursFareStrategy implements this interface:

```
public class PeakHoursFareStrategy implements FareStrategy {
    // 50% higher during peak hours
    private static final BigDecimal PEAK_HOURS_MULTIPLIER = new
    // Base fee.
    private BigDecimal baseFee;

    public PeakHoursFareStrategy() {}

    @Override
    public BigDecimal calculateFare(Ticket ticket, BigDecimal inputFare) {
        BigDecimal fare = inputFare;
        if (isPeakHours(ticket.getEntryTime())) {
            fare = fare.multiply(PEAK_HOURS_MULTIPLIER);
        }
        return fare;
    }

    private boolean isPeakHours(LocalDateTime time) {
        int hour = time.getHour();
        return (hour >= 7 && hour <= 10) || (hour >= 16 && hour <= 19);
    }
}
```

calculateFare(Ticket ticket, BigDecimal inputFare):

- Multiplies the input fare by 1.5 if the entry time falls within peak hours. Otherwise, it leaves it unchanged.
- Adjusts the fare for high-demand periods, increasing costs during busy times.

isPeakHours(LocalDateTime time): Checks if the given time's hour is within peak ranges.

The FareCalculator class uses these strategies:

```
public class FareCalculator {
    private final ListFareStrategy fareStrategies;

    public FareCalculator(ListFareStrategy fareStrategies) {
        this.fareStrategies = fareStrategies;
    }

    public BigDecimal calculateFare(Ticket ticket) {
        BigDecimal fare = BigDecimal.ZERO;
        for (FareStrategy strategy : fareStrategies) {
            fare = strategy.calculateFare(ticket, fare);
        }
        return fare;
    }
}
```

FareCalculator(ListFareStrategy fareStrategies): Initializes with a list of strategies, setting up the rules to apply during fare calculation.

calculateFare(Ticket ticket): Starts with a zero fare, iterates through each strategy in the list, and applies their rules in sequence to build the final fare.

Implementation choice: We implement FareCalculator using a ListFareStrategy to hold strategies, enabling the sequential application of multiple rules (e.g., base fare followed by peak adjustment). We choose List over an array or Set because it preserves order. Strategies like BaseFareStrategy must be applied before PeakHoursFareStrategy for correct fare calculation. A Set can prevent duplicates but loses order, while an array maintains a fixed size, limiting flexibility.

ParkingLot Code

The ParkingLot class acts as a facade, providing a simple interface for clients to interact with the parking lot system while delegating complex tasks to ParkingManager and FareCalculator. It relies on ParkingManager for spot allocation and FareCalculator for pricing, managing the flow of vehicles through entry and exit operations.

Here's the implementation of the ParkingLot class:

```
public class ParkingLot {
    // Manages parking spots and vehicle assignments
    private ParkingManager parkingManager;
    // Calculates fare for parking sessions
    private FareCalculator fareCalculator;

    public ParkingLot(ParkingManager parkingManager, FareCalculator fareCalculator) {
        this.parkingManager = parkingManager;
        this.fareCalculator = fareCalculator;
    }

    // Method to handle vehicle entry into the parking lot
    public Ticket enterVehicle(Vehicle vehicle) {
        // Find a suitable spot for the vehicle
        ParkingSpot spot = parkingManager.parkVehicle(vehicle);

        if (spot != null) {
            // Create ticket with entry time
            Ticket ticket = new TicketFactory.createTicket(1
            return ticket;
        } else {
            return null; // No spot available
        }
    }

    // Method to handle vehicle exit from the parking lot
    public void leaveVehicle(Ticket ticket) {
        // Ensure the ticket is valid and the vehicle hasn't already left
        if (ticket != null && ticket.getExitTime() == null) {
            // Calculate exit time
            ticket.setExitTime(LocalDateTime.now());

            // Delegate unparking logic to ParkingManager
            parkingManager.unparkVehicle(ticket.getVehicle());

            // Calculate the fare
            BigDecimal fare = fareCalculator.calculateFare(ticket);
        } else {
            // Invalid ticket or vehicle already exited.
        }
    }
}
```

enterVehicle(Vehicle vehicle): Coordinates vehicle entry by requesting a parking spot from ParkingManager. It then generates a Ticket with a unique ID, vehicle, parking spot, and current entry time.

leaveVehicle(Ticket ticket): Manages vehicle exit by setting the exit time, frees the parking spot via ParkingManager, and calculates the fare with FareCalculator.

Deep Dive Topics

In this section, we'll cover common follow-up questions interviewers may ask about the parking lot system. These are important topics that interviewers might expect you to explore in detail.

Adding a New Parking Spot Type

The parking lot system is designed to support multiple parking spot types (e.g., CompactSpot, RegularSpot, OversizedSpot). However, there may be a need to introduce a new type, such as a handicapped parking spot, to accommodate specific requirements. The challenge is to extend the system efficiently without modifying existing classes, adhering to the Open-Closed Principle (open for extension, closed for modification).

To achieve this, we can introduce a new HandicappedSpot class that implements the existing ParkingSpot interface. This approach ensures smooth integration with the system's spot allocation and management logic, as ParkingManager already relies on the ParkingSpot interface for handling spots.

ParkingSpot with HandicappedSpot class

Below is the implementation of the HandicappedSpot class.

```
public class HandicappedSpot implements ParkingSpot {
    private int spotNumber;
    private Vehicle vehicle;

    public HandicappedSpot(int spotNumber) {
        this.spotNumber = spotNumber;
        this.vehicle = null;
    }

    @Override
    public int getSpotNumber() {
        return spotNumber;
    }

    @Override
    public boolean isAvailable() {
        return vehicle == null;
    }

    @Override
    public void occupy(Vehicle vehicle) {
        if (isAvailable()) {
            this.vehicle = vehicle;
        } else {
            // Spot is already occupied.
        }
    }

    @Override
    public void vacate() {
        this.vehicle = null;
    }

    @Override
    public VehicleSize getSize() {
        return VehicleSize.MEDIUM;
    }
}
```

Faster Parking Spot Management

The mapping we currently have is one-way: from vehicle to ParkingSpot. This allows us to quickly find the parking spot assigned to a specific vehicle. But what if we want to find which vehicle is parked in a specific spot? Without a reverse mapping, we would need to search through all parking spots, which isn't efficient. Can we do better?

We can enhance this by introducing another HashMap, called spotToVehicleMap, to track the reverse mapping from ParkingSpot to Vehicle.

With this approach, we use two HashMaps:

- vehicleToSpotMap: Tracks the parking spot for each vehicle.
- spotToVehicleMap: Tracks the vehicle parked in each spot.

Below is the updated ParkingManager class:

```
public class ParkingManager {
    private List<ParkingSpot> availableSpots, List<ParkingSpot> availableSpots;
    private Map<Vehicle, ParkingSpot> vehicleToSpotMap;
    private Map<ParkingSpot, Vehicle> spotToVehicleMap;

    // Create Parking Manager based on a given map of available spots
    public ParkingManager(Map<VehicleSize, List<ParkingSpot>> availableSpots) {
        this.availableSpots = new ArrayList<>();
        this.vehicleToSpotMap = new HashMap<>();
        this.spotToVehicleMap = new HashMap<>();
    }

    public ParkingSpot findSpotForVehicle(Vehicle vehicle) {
        // No change in the method
    }

    public ParkingSpot parkVehicle(Vehicle vehicle) {
        ParkingSpot spot = findSpotForVehicle(vehicle);
        if (spot != null) {
            spot.occupy(vehicle);
            // Update both mappings
            vehicleToSpotMap.put(vehicle, spot);
            spotToVehicleMap.put(spot, vehicle);
            // Remove the spot from the available list
            availableSpots.get(spot.getSize()).remove(spot);
            return spot; // Parking successful
        }
        return null; // No spot found for this vehicle
    }

    public void unparkVehicle(Vehicle vehicle) {
        ParkingSpot spot = vehicleToSpotMap.remove(vehicle);
        if (spot != null) {
            spotToVehicleMap.remove(spot);
            spot.vacate();
            availableSpots.get(spot.getSize()).add(spot);
        }
    }

    // Find vehicle's parking spot
    public ParkingSpot findSpotForSpot(ParkingSpot spot) {
        return spotToVehicleMap.get(spot);
    }
    public Vehicle findSpotByVehicle(ParkingSpot spot) {
        return spotToVehicleMap.get(spot);
    }
}
```

Implementation Benefits: The bidirectional mapping in ParkingManager enhances performance by adding a spotToVehicleMap alongside the vehicleToSpotMap, enabling O(1) lookups from a vehicle to a parking spot and vice versa. This eliminates the need to iterate through all parked vehicles to identify the one in a given parking spot. It's especially efficient in large parking lots, where such iterations can be expensive.

With this enhancement explored, let's summarize the key takeaways.

Wrap Up

In this chapter, we gathered requirements for the Parking Lot system through detailed questions and answers. We identified the core objects involved, designed the class structure, and implemented the system's key components.

A key takeaway from this design is the value of modularity and clear separation of concerns. Each component, such as Vehicle, ParkingSpot, ParkingManager, and FareCalculator, handles a distinct responsibility, keeping the system maintainable and open to future enhancements.

Our design choices, like using ParkingLot as a facade to coordinate operations or employing the FareStrategy interface for flexible pricing, emphasize simplicity and adaptability. An extension could involve introducing an embedding spot allocation and fare logic directly in ParkingLot, might reduce the number of classes but could complicate scalability by overloading a single class with multiple responsibilities. In an interview, reflecting on these decisions and explaining their benefits showcases your ability to balance trade-offs in object-oriented design.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Further Reading: Strategy and Facade Design Patterns

This section gives a quick overview of the design patterns used in this chapter. It's helpful if you're new to these patterns or need a refresher to understand the design choices better.

Strategy design pattern

The Strategy pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one in a separate class, and allows their objects to be interchangeable.

In the parking lot design, we have used the Strategy pattern to encapsulate pricing rules in the FareStrategy interface (e.g., BaseFareStrategy, PeakHoursFareStrategy), allowing FareCalculator to switch between rules dynamically without altering its core logic.

To illustrate the Strategy pattern in another domain, the following example uses an e-commerce payment system.

Problem

Imagine you're developing an e-commerce application that offers various payment methods, such as credit cards, PayPal, and bank transfers. Initially, you might implement each payment method directly within the checkout process. However, as the application grows, this approach can lead to a monolithic design where the payment processing logic becomes tightly coupled with the checkout system. This tight coupling makes it challenging to add new payment methods or modify existing ones without changing the core checkout code, which increases the risk of introducing bugs and makes the system harder to maintain.

Solution

To address this issue, the Strategy design pattern can be employed. This pattern suggests encapsulating each payment algorithm in a separate class, known as a strategy, and making them interchangeable. The main application, referred to as the context, maintains a reference to a strategy object and delegates the payment processing to this object. This design allows the application to switch between different payment methods, without modifying the core checkout logic.

Strategy design pattern class diagram

When to use

The Strategy design pattern is particularly useful in scenarios:

- When an application needs to select different algorithms or behaviors at runtime based on specific conditions, the Strategy pattern is a great fit.
- When a class is cluttered with conditional statements to choose between different algorithm variations, the Strategy pattern simplifies things. It moves each algorithm into its own class, with all classes implementing the same interface. This lets the original object delegate the task to the right class without complex conditionals.
- Use the Strategy pattern to keep your client's business logic separate from the implementation details of the tasks.

Facade design pattern

The Facade pattern is a structural design pattern that provides a simple interface to a complex subsystem, such as a library, framework, or set of classes. It simplifies how clients interact with the system by hiding its underlying complexity.

In the parking lot design, the Facade pattern is used in the ParkingLot class, which streamlines client interactions by managing tasks like vehicle entry, spot assignment, and fee calculation, delegating to subsystems such as ParkingManager and FareCalculator.

To illustrate the Facade pattern in another domain, the following example uses a home theater system.

Problem

Imagine you're setting up a home theater system with multiple components, such as a DVD player, projector, sound system, and lights. To watch a movie, you must turn on each component, adjust settings, and synchronize them. This process is complex, requiring users to understand each component's working. As the system grows, adding new devices (e.g., a streaming device) increases complexity, making it harder to use the system efficiently.

Solution

The Facade pattern addresses this by introducing a simple interface, the facade, that encapsulates the subsystem's complexity. For the home theater, a HomeTheaterFacade class could provide methods like watchMovie(), which internally manages all components (e.g., turning on the projector, setting the sound system). Clients interact only with the facade, which delegates tasks to the subsystem, simplifying usage.

Facade design pattern class diagram

When to use

The Facade design pattern is particularly useful in scenarios:

- When a subsystem is complex, with multiple components or interactions, and you want to provide a simpler interface for clients.
- When you want to layer a system into subsystems, but still offer a unified entry point for common operations.

Mark as Complete

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.